
Deep Learning for Mortgage Risk

A didactic, build-it-from-the-ground-up walkthrough

of the dissertation: replicating Sirignano, Sadhwani & Giesecke (2021)
on the public Fannie Mae loan-performance data

*From the raw loan tape, through the data pipeline,
to the neural network, and all the way to pool-level MBS predictions.*

This is a teaching companion, not the dissertation itself. It is meant to be read straight through: every term is explained the first time it appears, the mathematics is built up slowly from logistic regression, and the two parts you asked to dwell on — the neural network and the pool-level results — are given the most room. Where the project still has open decisions, they are flagged honestly rather than papered over.

A companion guide

Contents

1	The whole thing in one page	3
2	Financial and economic prerequisites: a primer	3
2.1	What a mortgage actually is	3
2.2	Fixed, adjustable, and the fine print	4
2.3	How a mortgage ends: the two competing fates	4
2.4	The borrower's two embedded options	5
2.5	The risk drivers: how a loan is sized up	5
2.6	The macro backdrop	6
2.7	Why prepayment is the hard one: seasoning and burnout	6
2.8	Securitisation and mortgage-backed securities	6
2.9	How the primer maps onto the model	7
3	The raw material: the Fannie Mae loan tape	7
3.1	What the data physically is	7
3.2	The fields that matter	8
4	From a loan's life to a prediction problem	9
4.1	The seven states	9
4.2	Where the seven states come from in the raw data	9
4.3	The empirical transition matrix — our humble benchmark	10
5	The data pipeline: turning 219 GB into something trainable	11
5.1	The one rule that drives the whole design	11
5.2	The six stages	12
5.3	Two subtle correctness rules	12
6	Feature engineering: the economics in the inputs	13
6.1	The refinancing incentive (and why it is so nonlinear)	13
6.2	Current LTV (using a house-price index)	14
6.3	Path-dependent delinquency counts (a recommendation)	14
6.4	Which features actually matter (evidence from the paper)	15
6.5	The standardisation discipline	16
7	The neural network, built from logistic regression up	16
7.1	Move 0: what the model must output	16
7.2	Move 1: the softmax turns numbers into probabilities	17
7.3	Move 2: logistic regression is softmax of a straight line	17
7.4	Move 3: from fixed features to <i>learned</i> features	17
7.5	Move 4: stacking layers	18
7.6	The activation: ReLU	18
7.7	Our concrete architecture	19
7.8	Why depth, and why one network for all transitions	20
8	Training: maximum likelihood is cross-entropy	20
8.1	The likelihood, and why it collapses to a clean sum	20
8.2	Negative log-likelihood = cross-entropy loss	21
8.3	The gradient that makes it all work	21
8.4	Backpropagation in one equation	22
8.5	Minibatch stochastic gradient descent	22

8.6	Keeping a flexible model honest: three guards against overfitting	23
8.7	The strictly temporal split (and our open cutoff question)	23
9	Loan-level results: does depth actually help?	24
9.1	Goodness of fit: the cross-entropy ladder	24
9.2	Discrimination by transition: AUC	25
9.3	What the paper actually found (the headline results)	26
9.4	What we realistically expect to replicate	26
10	Pool-level prediction: the result that matters most	27
10.1	Why pools, and why investors care	27
10.2	The crux: a pool prediction is a sum, not a new model	27
10.3	The roll-forward, explained slowly	27
10.4	Building the pools	29
10.5	The headline picture: observed vs. predicted	29
10.6	The full distribution, via Monte Carlo on rates	30
10.7	Why the linear model is biased and the network is not	32
11	A non-neural challenger: gradient-boosted trees	32
11.1	The question GBT is built to answer	33
11.2	What a decision tree is	33
11.3	From one tree to a boosted ensemble	33
11.4	Two flexibilities, opposite biases	35
11.5	Why GBT, and not the other candidates	35
11.6	The gap in the literature it fills	36
11.7	Where it plugs in: reuse, not rebuild	36
11.8	The one extra step the net did not need: calibration	37
11.9	What each outcome would mean (pre-registered)	37
11.10	Its contribution to the dissertation	38
12	Our approach: what transfers, what changes, what is open	38
12.1	What transfers wholesale	38
12.2	What we change	38
12.3	What is genuinely open (good questions for the advisor)	39
13	How it all fits together	39

1 The whole thing in one page

Before any detail, here is the entire dissertation in a single breath, so that every later section has a place to hang.

The question. How do mortgage borrowers behave — when do they fall behind, recover, default, or pay off early — and can we predict those behaviours far more accurately by letting the data, rather than a pre-chosen formula, dictate the shape of the relationships?

The method. We model, for every loan in every month, the probability that the loan moves to each of seven possible states next month. That is a *classification* problem with seven classes. We solve it first with the textbook tool (logistic regression) and then with its powerful nonlinear cousin (a deep neural network), and we compare them.

The pay-off. The headline of the paper we are replicating is that the nonlinearities matter enormously — not just for predicting a single loan, but for predicting the behaviour of a *pool* of a thousand loans, which is exactly what a mortgage-backed security (MBS) is built on. The linear model is systematically biased at the pool level; the neural network is not. That is the result we most want to reproduce.

Our twist. The original paper used a proprietary CoreLogic dataset of 120+ million loans. We cannot get that data. Instead we rebuild the whole machine on the *public* Fannie Mae Single-Family Loan Performance dataset. The data source changes; the methodology transfers almost entirely.

The single sentence to remember

We are building a *loan-level* seven-state monthly transition model with a neural network, and then — with *no separate pool model* — we add up the loan-level predictions to forecast what a whole mortgage pool will do.

Why “no separate pool model” is the crux

A pool’s behaviour is just the sum of its loans’ behaviours. If you can predict each loan well, you can predict the pool by addition. This sounds trivial, but it is the quiet engine of the entire pool-level result: it means every drop of accuracy you win at the loan level is inherited, for free, at the pool level — and every bias you carry is inherited too. We return to this at length in Section 10.

2 Financial and economic prerequisites: a primer

The rest of this document assumes a working vocabulary of mortgage finance. This section supplies it from scratch, for a reader who is comfortable with mathematics and machine learning but has never priced a loan. A reader who already knows what prepayment, LTV, and an agency MBS are can skim to Section 3; everything here reappears in context later, so nothing is lost by moving quickly.

2.1 What a mortgage actually is

A **mortgage** is a loan used to buy property, *secured* by that property: if the borrower stops paying, the lender can seize and sell the home. It has three defining numbers at origination — the *principal* (amount borrowed), the *interest rate*, and the *term* (length, classically 30 years) — and the borrower repays it in equal monthly instalments.

Amortisation and the level payment

The fixed monthly payment is split each month between *interest* (the rent on the money still owed) and *principal* (paying down the debt). Early on, almost all of the payment is interest because the balance is large; over time the balance shrinks and the principal share grows. This gradual pay-down is called *amortisation*, and the amount still owed at any month is the *unpaid principal balance* (UPB).

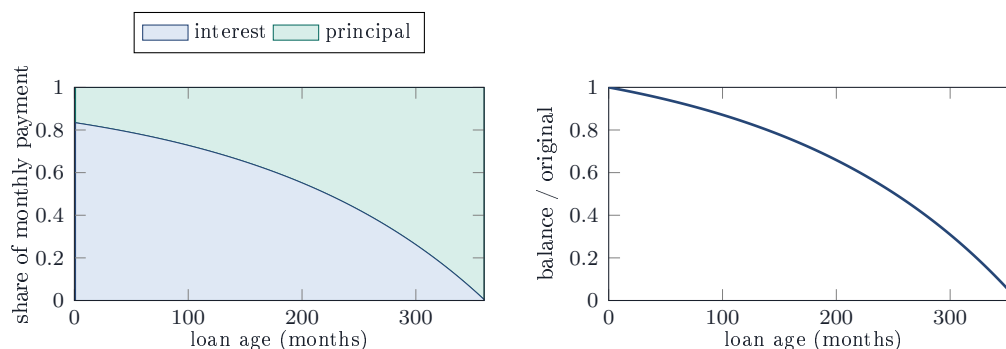


Figure 1: A 30-year fixed loan amortising (schematic). **Left:** the constant monthly payment is mostly interest at first and mostly principal near the end; the crossover comes surprisingly late (well past the half-way point in years). **Right:** the remaining balance (UPB) therefore falls slowly at first and faster later. The model’s *Loan Age*, *Current Actual UPB*, and *Remaining Months to Maturity* fields all track positions along these curves.

2.2 Fixed, adjustable, and the fine print

Not all mortgages are the plain 30-year fixed loan above. The features that matter for our data are:

- **FRM vs. ARM.** A *fixed-rate mortgage* keeps one rate for life. An *adjustable-rate mortgage* starts with a fixed period and then resets periodically to a market index. ARMs often begin with a low “teaser” rate that later jumps — a common trigger for refinancing.
- **Interest-only loans** defer principal for a while; **prepayment penalties** charge the borrower for paying off early; **balloon** loans owe a lump sum at the end.
- **Conforming vs. jumbo.** Loans small enough and standard enough to be bought by the government-sponsored enterprises are “conforming” — the Fannie Mae data is exactly this conforming population.

Why the product type shows up in the model

Each of these features bends borrower behaviour. An ARM about to reset, or a loan whose prepayment penalty just expired, suddenly becomes far likelier to prepay — which is why the data carries **Amortization Type**, **Interest-Only Indicator**, and related fields, and why the loan-age “spikes” in prepayment (Section 6) line up with reset and penalty-expiry dates.

2.3 How a mortgage ends: the two competing fates

A mortgage can leave the books in two very different ways, and almost the entire dissertation is about telling them apart.

- **Prepayment** — the borrower pays the loan off *early*, almost always by refinancing into a cheaper loan or selling the house. The lender gets their principal back, but loses the future interest they were counting on.
- **Default** — the borrower stops paying. This unfolds through a *delinquency ladder*: 30, then 60, then 90+ days past due, then *foreclosure* (the legal seizure of the home), then *REO* (“real-estate owned,” the lender now holds and must sell the property).

Importantly, borrowers can *cure* — a loan 60 days late can return to current — so the ladder is climbed in both directions, not a one-way slide.

Competing risks

Prepayment and default are *competing risks*: a loan can only exit once, so the two terminal outcomes compete for each loan. A model of mortgage behaviour must therefore predict *both* simultaneously rather than studying one in isolation — which is precisely why the seven-state transition model (Section 4) treats **prepaid**, **foreclosure**, and **REO** as distinct absorbing outcomes in one framework.

2.4 The borrower’s two embedded options

There is an elegant way to see why these two fates behave so differently. Holding a mortgage is, for the borrower, like holding two financial *options*.

- The **prepayment option** is a *call* on the loan: the borrower may “buy back” their debt at par whenever they like. It pays to exercise when market rates fall below the loan’s rate — refinance, and capture the difference. This is the refinancing incentive.
- The **default option** is a *put* on the house: the borrower may “sell” the property to the lender by handing over the keys. It is in the money when the home is worth less than the loan — the borrower is *underwater*.

Why the option view matters — and why it is not the whole story

The option lens, developed in the classic mortgage literature (e.g. Schwartz & Torous, 1989; Deng, Quigley & Van Order, 2000), says interest rates should drive prepayment and house prices should drive default. That is broadly true and motivates which variables we feed the model. But real borrowers do *not* exercise these options with textbook precision: they prepay late, default reluctantly, and respond to credit scores, loan size, and personal circumstances in tangled, nonlinear ways. The gap between the clean option theory and messy reality is exactly the territory a flexible deep-learning model is built to map.

2.5 The risk drivers: how a loan is sized up

A handful of standardised numbers summarise a loan’s risk, and they recur as model features:

- **FICO** — the borrower’s credit score (roughly 300–850); higher means a stronger repayment history.
- **LTV / CLTV** — *loan-to-value* is the loan divided by the home’s value; *combined* LTV adds any second liens. High LTV means little equity cushion.
- **DTI** — *debt-to-income*, the share of the borrower’s income consumed by debt payments; high DTI means a stretched borrower.

Original vs. current LTV, and “underwater”

LTV at origination is fixed, but the one that drives behaviour is *current* LTV: today’s balance over today’s home value. As prices rise the borrower builds *equity* (current LTV falls) and can refinance easily; as prices fall they may go *underwater* (current LTV above 100%), unable to sell or refinance and likelier to default. Reconstructing current LTV from a house-price index is one of the key feature-engineering steps in Section 6.

2.6 The macro backdrop

Borrowers do not act in a vacuum; a few economy-wide variables move everyone at once:

- **Interest rates.** The prevailing mortgage rate (we use the Freddie Mac PMMS series) sets the refinancing incentive. When it drops, prepayment surges.
- **House prices.** A house-price index (we use the FHFA HPI) updates each home’s value and hence current LTV and equity, governing default pressure.
- **Unemployment.** Job loss is a classic trigger of delinquency; local unemployment is a standard default predictor.

Common factors create correlation

Because every loan in a region shares the same rates, prices, and unemployment, their fates are *correlated*: when rates fall, a whole pool tends to prepay together. This shared exposure is what makes pool-level prediction (Section 10) more than a trivial sum — and, as we will see, it is also why a mis-specified linear model’s errors reinforce rather than cancel.

2.7 Why prepayment is the hard one: seasoning and burnout

Seasoning and burnout

Seasoning is the tendency of prepayment to ramp up over a loan’s first few years rather than starting at full speed — new borrowers rarely refinance immediately. *Burnout* is the opposite-end effect: after a refinancing wave, the borrowers left in a pool are the ones who *didn’t* act, so the pool prepays less even when incentives return. Both are *path-dependent* — they depend on the history a loan has lived through, not just today’s numbers.

These effects, layered on top of the already S-shaped response to incentive, make prepayment the most violently nonlinear of all the transitions — and the place where the deep model most outperforms the linear one, at both the loan and pool level.

2.8 Securitisation and mortgage-backed securities

Finally, the reason any of this is worth millions of dollars: most of these loans are not held by their originators but *securitised*.

Securitisation, pass-through, agency MBS, GSE

Securitisation bundles many loans into a pool and sells investors claims on the pool’s combined cash flows. The simplest claim is a *pass-through*: borrowers’ monthly principal and interest flow through to investors pro rata. When the pool is guaranteed by a *government-*

sponsored enterprise (GSE) such as Fannie Mae, the result is an *agency mortgage-backed security* (MBS). The GSE guarantee removes most *credit* risk for the investor — but, crucially, *not prepayment risk*.

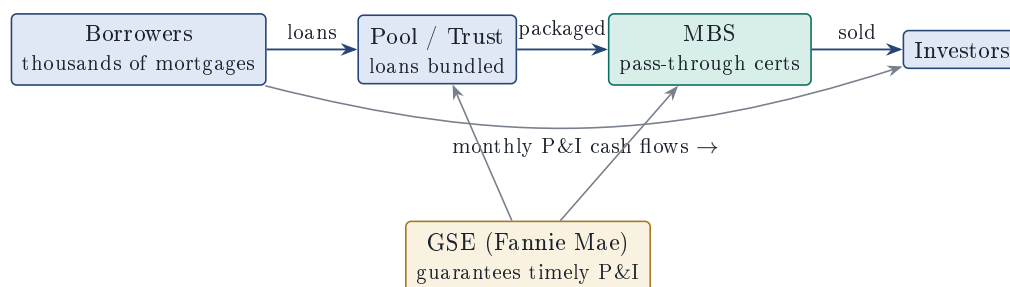


Figure 2: The securitisation chain. Loans are pooled and sold to investors as pass-through MBS; borrowers’ monthly principal and interest flow up to those investors. The GSE guarantee makes the credit risk the investor’s least worry — leaving prepayment as the dominant risk they must forecast, which is what makes accurate pool-level prepayment prediction so valuable.

Prepayment risk and “negative convexity”

An MBS investor is, in effect, *short* the borrowers’ prepayment option. When rates fall, exactly when the investor would most like to keep their high-yielding bond, borrowers refinance and hand the principal back early — to be reinvested at the new, lower rate. When rates rise, prepayments slow and the investor is stuck holding a below-market bond for longer. Heads the investor loses a little, tails they lose a little: this asymmetry is called *negative convexity*, and it is why mispricing pool-level prepayment translates directly into mispriced MBS.

2.9 How the primer maps onto the model

The whole primer, in one correspondence

The mortgage’s possible fates become the model’s **seven states**; the risk drivers and macro variables become its **features**; the embedded-option intuition tells us *which* features matter for *which* transition; and the securitised pool is the **pool-level target** whose prepayment the model must forecast. Everything in this primer is, in the end, either a label, a feature, or the thing we are ultimately trying to price.

3 The raw material: the Fannie Mae loan tape

3.1 What the data physically is

The Fannie Mae Single-Family Loan Performance (SFLP) dataset is a *loan-month panel*. Read that phrase carefully, because it shapes everything.

Loan-month panel

A “panel” tracks many subjects over time. Here each *subject* is one mortgage, and we observe it *every month* it is alive. So the unit of a single row is not “a loan” but “a loan in a particular month” — a *loan-month*. One thirty-year mortgage that survives to maturity

contributes up to 360 rows, one per monthly report.

Concretely, the raw data we hold is:

- about **219 GB** spread over 26 quarterly CSV files (the largest single file is roughly 25 GB);
- **pipe-delimited** (| between fields) with **no header row** — column identity is purely *positional*, so the order of the 113 fields *is* the contract;
- **113 fields** per row, with dates encoded as MMYYYY (e.g. 112013 means November 2013);
- partial and a little dirty: several quarters are missing, and at least three files are truncated downloads (one “quarter” contains only 15 distinct loans).

Why the size is a real obstacle, not a footnote

A 25 GB file will not fit in a laptop’s memory, and the naive `pandas.read_csv` on it will crash the machine. The full panel is on the order of one to two *billion* loan-months. So a surprising amount of the dissertation’s engineering effort goes into *never loading the whole thing at once*. That constraint — not the statistics — is what dictates the architecture of the pipeline in Section 5.

It helps to see our raw material beside the paper’s, since the replication’s credibility rests on the data being comparable in spirit even though the source differs.

Table 1: *The two datasets at a glance. Ours is public and conforming; the paper’s was proprietary and spanned subprime as well as prime. The methodology transfers; the population and provenance differ.*

	This dissertation	Original paper
Source	Fannie Mae SFLP (public)	CoreLogic (proprietary)
Population	prime, conforming	prime <i>and</i> subprime
Scale	~1–2 billion loan-months; 219 GB raw, 113 fields	~3.5 billion loan-months; ~118 million mortgages
Span	~2000–2025 (includes COVID)	1995–2014
Macro covariates (first pass)	rate (PMMS) + price (FHFA HPI); fuller macro deferred	rich macro incl. zip-level prices, unemployment
Target	seven-state monthly transition	seven-state monthly transition

3.2 The fields that matter

Of the 113 columns, most are post-default servicing and expense fields we will not use as predictors. The ones that carry the modelling signal fall into a few families:

- **Identity and time:** **Loan Identifier** (stable across a loan’s months — this is the key that lets us follow a loan), and **Monthly Reporting Period** (the time index).
- **Origination characteristics** (fixed at the start): original interest rate, original balance (UPB), original term, original loan-to-value (LTV) and combined LTV, debt-to-income (DTI), borrower and co-borrower FICO credit scores, loan purpose, property type, occupancy, state, MSA, 3-digit zip.
- **Contemporary state** (changes month to month): current interest rate, current actual balance, loan age, remaining months to maturity, and the two fields that drive our target — **Current Loan Delinquency Status** and **Zero Balance Code**.

UPB

Unpaid Principal Balance — how much of the loan the borrower still owes. The *current actual* UPB falling faster than the scheduled amortisation is a signal of partial prepayment (“curtailment”); the UPB going to zero is the loan terminating.

4 From a loan’s life to a prediction problem

4.1 The seven states

We follow the paper in collapsing a mortgage’s status into one of **seven states**:

`current` → `30 dpd` → `60 dpd` → `90+ dpd` → `foreclosure` → `REO` and `prepaid`

where “dpd” is days-past-due. A loan does *not* march down this line in order: it hops around. A borrower can miss a payment (`current` → `30 dpd`), catch up the next month (`30 dpd` → `current`), and pay the loan off years later (`current` → `prepaid`).

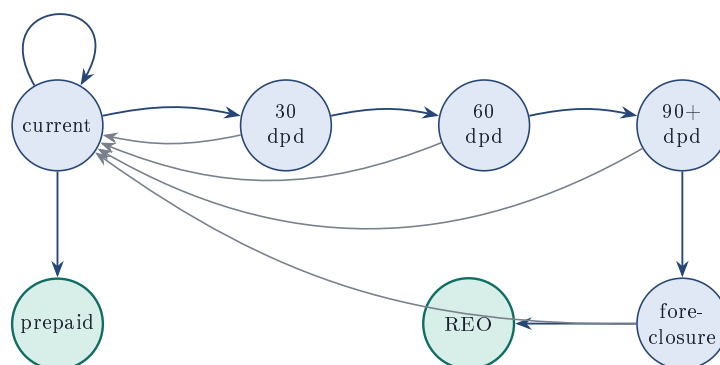


Figure 3: The seven-state machine. Blue arrows are “getting worse” transitions, grey arrows are “curing” back toward `current`, and the self-loop is staying current. The two teal nodes, `prepaid` and `REO`, are absorbing: once entered, the loan leaves the sample. Note that `current` can jump straight to `prepaid` (a refinance or sale) without passing through delinquency.

REO and prepaid

`REO` is “real-estate owned” — the lender has taken the property after foreclosure. `prepaid` means the loan ended early by being paid in full (usually a refinance or a home sale). `REO` and `prepaid` are treated as *absorbing*: once a loan reaches them it leaves the sample, because there is nothing more to predict.

4.2 Where the seven states come from in the raw data

The data does not hand us a tidy “state” column; we derive it from two fields.

- **Current Loan Delinquency Status** (a string 00, 01, 02, 03, ...): 00 is current, 01 is 30 days late, 02 is 60, 03+ is 90+ days late.
- **Zero Balance Code**: this is *blank for every month except the loan’s final, terminating month*. When present it tells us *how* the loan ended — code 01 = prepaid/matured, 09 = REO/deed-in-lieu, and a cluster of codes (02, 03, 15, 97) that map to a credit-event **foreclosure**.

So the rule, per loan-month, is: *if a Zero Balance Code is present, the state is the terminal state it encodes; otherwise the delinquency status governs.*

The supervised label is a transition, not a state

For each loan-month we record the current state U_t and the same loan’s state the following month, U_{t+1} (we call it **state_next**, obtained with a window function that looks one month ahead within each loan). The pair (U_t, U_{t+1}) is what the model learns to predict. The label is the *next* state given the current one and everything we know today.

month t	state U_t	state_next U_{t+1}
Jan	current	current
Feb	current	30 dpd
Mar	30 dpd	current
Apr	current	prepaid

state_next is next row's state

Figure 4: One loan’s panel rows. The label **state_next** for each month is simply that same loan’s **state** in the following month — a one-row shift. Because this peeks one month into the future, the leakage-safe time mask in Section 5 must be strict. The last row of an alive-but-current loan would have an empty **state_next**: that is the censored case.

Censoring: the loans that simply “run out”

Some loans are still alive and current on the last date in our data. They have no **state_next** — not because anything happened, but because the data stops. These are *right-censored*, and we flag them. It would be a mistake to treat “no observed next month” as “the loan disappeared.” Keeping a **censored** flag lets the training code handle them correctly instead of inventing a fake terminal event.

4.3 The empirical transition matrix — our humble benchmark

Before any neural network, there is an almost embarrassingly simple model we must beat. Count, across the whole dataset, how often a loan in state u moved to state v next month, and divide by how often it was in state u at all:

$$\hat{\mathbb{P}}_{\text{emp}}(v | u) = \frac{\#\{\text{loan-months: } u \rightarrow v\}}{\#\{\text{loan-months in } u\}}.$$

Stack these into a 7×7 matrix and you have the *empirical transition matrix*. It predicts the next state using *only* the current state — it ignores FICO, rates, balance, everything.

Its quality as a model is measured by the same loss we will use for the networks — the negative log-likelihood it assigns to the realised transitions:

$$\text{loss}_{\text{emp}} = -\frac{1}{M} \sum_{(u \rightarrow v) \text{ observed}} \log \hat{\mathbb{P}}_{\text{emp}}(v | u), \quad (1)$$

where M is the number of transitions. Every richer model must drive this number down.

Why bother with a benchmark this dumb?

Because a model that cannot beat “just use the historical frequencies” has learned nothing useful from all those covariates. The empirical matrix sets the bar. We will measure every

model — logistic regression, shallow net, deep net — by how far below this benchmark’s loss they can push (Section 8). Most of the mass sits on `current` → `current`, so a good model must earn its keep on the rare, costly transitions: into delinquency, foreclosure, and prepayment.

How lopsided the classes really are

“Most of the mass” undersells it. In a healthy prime book the overwhelming majority of loan-months are `current` → `current`, while the credit-event transitions (into foreclosure and REO) are well under 1% of rows — the events that matter most for cashflows are the rarest in the data. This is why the pipeline samples in a *stratified* way: it keeps every scarce foreclosure and REO transition and down-samples the torrent of `current` → `current`, then corrects for that thinning with the importance weights of Section 8. After thinning, the training export is on the order of tens of millions of rows (~74 M) over roughly thirty features — small enough to train on, balanced enough to learn the rare events. The imbalance is not a nuisance to be smoothed away; it is the reason the rare-transition accuracy in the results tables is the number to watch.

5 The data pipeline: turning 219 GB into something trainable

This section is the engineering heart of the project. It earns its length because if the pipeline is wrong — if it leaks the future, double-counts a loan, or silently mixes a truncated file into training — then every clever thing downstream is built on sand.

5.1 The one rule that drives the whole design

Never load a full quarter into memory

Every stage operates on *bounded chunks* — a row-group, a batch, one quarter — or pushes the heavy lifting down into a database engine that streams from disk. No stage ever calls a full `.collect()` on the whole panel.

The stack chosen to honour that rule is **Parquet** + **DuckDB** + **Polars**:

- **Parquet** (with `zstd` compression) is a *columnar* file format. Storing by column means a query that needs 6 of 113 columns reads about 5% of the bytes, and the dataset compresses roughly 4–8× — the 219 GB raw lands in ~30–50 GB.
- **DuckDB** runs SQL *directly over* the Parquet files, reading only the columns and rows a query touches and spilling to disk when memory is tight. You can `SELECT ... WHERE acq_quarter='2020Q4'` without loading anything else.
- **Polars** is a DataFrame library whose *lazy* `scan/sink` operations stream through data in bounded memory and parallelise across cores.

Columnar vs row storage

A spreadsheet stores row by row: all of Alice’s fields, then all of Bob’s. Columnar storage keeps all the FICO scores together, all the balances together, and so on. If you only want FICO and balance, columnar lets you read just those two stripes off disk and skip the other 111 fields entirely. For analytics over a few columns of a huge table, this is the

difference between minutes and hours.

5.2 The six stages

The pipeline is six *idempotent, per-quarter* stages — re-running skips work already done, so a crash mid-run is recoverable.

1. **Inventory & integrity audit.** Before trusting anything, catalogue every file: row count, distinct loans, time span, field count. Quarantine the truncated files. *This is non-negotiable and goes first* — you do not model against a dataset whose coverage you have not mapped.
2. **Stream CSV → Parquet.** Convert each raw CSV to compressed Parquet in bounded chunks, keeping a faithful string-typed copy (so this stage never fails on a dirty value).
3. **Clean & standardise.** Cast types (MMYYYY → date, floats → 32-bit, sentinels like FICO 9999 → null), project down to the columns the model needs, and add calendar keys (`period_ym`, `orig_ym`) and *missingness indicators*.
4. **Build the transition panel.** Derive `state`, `state_next` and `censored` with a per-loan ordered window, plus a `shard` key (see below).
5. **Views & stratified sampling.** Expose the panel through DuckDB and draw RAM-sized samples that *keep all the rare transitions* (foreclosure, REO) and down-sample the overwhelming `current` → `current` mass.
6. **QA & reconciliation.** Prove each stage preserved row counts, audit null rates, print the 7×7 transition counts, and confirm no duplicate loan-months.

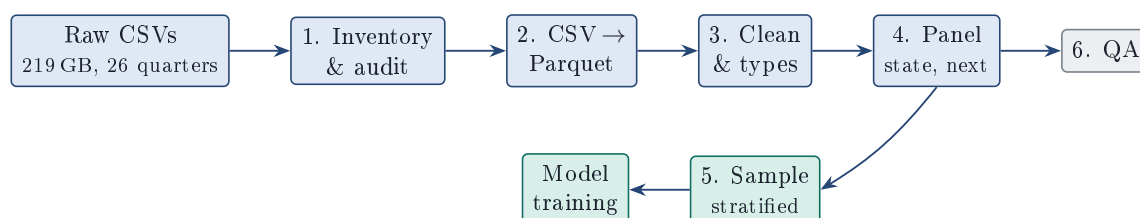


Figure 5: The six-stage pipeline. Raw immutable CSVs flow left to right into a compressed, queryable Parquet lake; Stage 5 draws RAM-sized stratified samples for the model. The governing constraint is that no arrow ever requires a full quarter in memory — everything streams in bounded chunks.

5.3 Two subtle correctness rules

Two rules deserve to be singled out because they are easy to get wrong and catastrophic when you do.

Rule 1: the leakage-safe time mask

Our label looks *one month into the future* (`state_next`). So if we want to train “on everything up to month T ,” we must filter on `period_ym` < T , *strictly*, not $\leq T$. Including month T itself would feed the model a label ($T+1$ ’s state) that lies past the cutoff — the future leaking into the past. A one-character slip (< vs $\leq$) silently inflates every backtest. This is the kind of bug that makes a model look brilliant and then fail in production.

Rule 2: shard by loan, never split a loan across train and test

We bucket each loan into one of N shards via a deterministic hash of its **Loan Identifier**: `shard = hash(loan_id) % N_SHARDS`. Because the hash is keyed on the loan, *every month of a given loan lands in the same shard*. That guarantees a loan never straddles train and test — which would let the model peek at the same borrower’s other months — and it gives us a cheap, reproducible way to stream random minibatches: shuffle shard order, load one shard (RAM-sized), shuffle within it, yield batches. (This is exactly the hash-table-to-folders trick the original paper describes for de-biasing minibatches.)

Why the empirical transition matrix needs the panel to be exactly right

The benchmark counts $u \rightarrow v$ transitions. If a single loan’s months were accidentally split across two shards, or a truncated file’s loans leaked in, those counts — and therefore the benchmark, and therefore every comparison against it — would be quietly wrong. The unglamorous QA stage is what lets you trust the headline numbers later.

6 Feature engineering: the economics in the inputs

Raw fields are not yet good predictors. The paper’s most important findings live in a few *engineered* features that encode mortgage economics. Three are worth understanding deeply, because they are where the nonlinearities the whole project is about actually appear.

6.1 The refinancing incentive (and why it is so nonlinear)

A borrower refinances — i.e. prepays the old loan — when the market rate has fallen enough below their loan’s rate to make it worthwhile. So the natural feature is the *rate incentive*:

$$\text{incentive}_t = (\text{the loan's note rate}) - (\text{current market mortgage rate}).$$

For the market rate we use the Freddie Mac **PMMS** (Primary Mortgage Market Survey), a long weekly series of the prevailing 30-year rate.

PMMS

Freddie Mac’s Primary Mortgage Market Survey: the standard published series for “the mortgage rate this week.” It is the yardstick against which a borrower’s own rate is high or low. Note the small irony of our project: the *loans* come from Fannie Mae, but the *rate series* comes from Freddie Mac — they are independent public sources.

The incentive curve is an S, not a line

You might guess prepayment rises smoothly with incentive. It does not. When incentive is negative (your rate is below market) almost nobody refinances — flat. As incentive turns positive there is a steep surge — people pounce on the savings. Then it *plateaus*: past a point, everyone who was going to refinance already has. The sensitivity of prepayment to incentive therefore changes in *both magnitude and sign* along the curve. A linear model, which assumes a single constant slope, is structurally incapable of fitting this — and prepayment is where the paper finds the strongest nonlinearities of all. This single picture is the moral motivation for using a neural network.

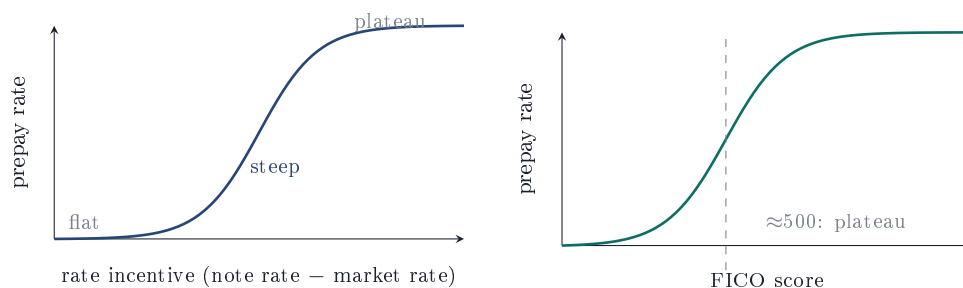


Figure 6: Two of the strong nonlinearities a linear model cannot capture (schematic, after the paper’s empirical figures). **Left:** prepayment versus refinancing incentive is S-shaped — flat, then a steep surge, then a plateau. **Right:** prepayment rises with FICO but levels off above roughly 500. A single slope cannot describe either curve.

Burnout

A pool that has already lived through a refinancing wave is left with the borrowers who *didn’t* refinance — the sluggish, the credit-impaired, the inattentive. So even at the same incentive, a “burned-out” pool prepays less. Burnout is a path-dependent feature: it depends on the history of incentives the loan has seen, not just today’s.

6.2 Current LTV (using a house-price index)

Original LTV is fixed at origination, but what matters for default and refinancing is the *current* loan-to-value: today’s balance over today’s home value. We do not observe each home’s price monthly, so we update it with the FHFA House Price Index (HPI):

$$\text{current LTV}_t \approx \frac{\text{current UPB}_t}{\text{original value} \times \frac{\text{HPI}_t}{\text{HPI}_{\text{orig}}}}.$$

FHFA HPI

The Federal Housing Finance Agency’s House Price Index measures how home prices have moved over time, available at national, state, and metro levels. Multiplying the original home value by the ratio of current-to-origination HPI gives an estimate of today’s value without an appraisal.

Why current LTV drives both default and prepayment

A borrower deep underwater (current LTV well above 100%) can neither sell at a profit nor easily refinance, and is likelier to default — the “put option” on the house is in the money. A borrower with lots of equity (low current LTV) refinances easily. So the same variable pulls two different transitions, which is part of why the paper models all transitions with *one* shared network rather than separate models.

6.3 Path-dependent delinquency counts (a recommendation)

The paper finds something striking: for predicting **current** $\rightarrow$ 30 dpd, the dominant variable is not FICO or LTV but *the number of times the borrower was 30 days late in the past 12 months*. Recent behaviour predicts near-future behaviour far better than static credit characteristics.

Engineer the rolling delinquency counts

Our current schema does not yet build these path-dependent counts (number of times 30/60/90+ days late in the trailing 12 months; number of times current). The paper’s results say they are among the most important predictors for the delinquency transitions. Adding them is one of the highest-value feature-engineering steps available to the replication.

6.4 Which features actually matter (evidence from the paper)

This is not just intuition — the paper measures it. For each transition it computes the *average absolute gradient* of the fitted network’s output with respect to each input: how much the predicted probability moves when that feature is nudged. The result for the **current** → **prepaid** transition (the paper’s Table 12, five-layer network) is worth reading closely, because almost every feature we engineered above appears near the top.

Table 2: Top features by average absolute gradient for **current** → **prepaid**, five-layer network (original paper, Table 12). Larger means the predicted prepayment probability is more sensitive to that feature.

Feature	Avg. gradient
Current outstanding balance	0.1878
Original loan balance	0.0856
Original interest rate	0.0503
Current rate — national mortgage rate (<i>the refinancing incentive</i>)	0.0478
Original rate — national mortgage rate	0.0463
Zip-code house-price change since origination (<i>drives current LTV</i>)	0.0386
Number of times 30 days delinquent in last 12 months (<i>path-dependent</i>)	0.0384
Number of times 60 days delinquent in last 12 months (<i>path-dependent</i>)	0.0362
Time since origination (<i>seasoning</i>)	0.0306
FICO score	0.0293
State unemployment rate	0.0214
LTV ratio	0.0116

Reading the importance table — it validates the whole section

Every engineered quantity from this section earns its place. The **refinancing incentive** (current rate minus the national rate) sits near the top; the **house-price change** that drives current LTV is just below it; the **path-dependent delinquency counts** we recommended building rank above FICO and far above the static LTV ratio; and **seasoning** (time since origination) is material. Two lessons stand out. First, *balance* variables dominate prepayment — larger loans have more dollars of interest to save by refinancing, so they respond most. Second, the celebrated static credit characteristics (FICO at 0.0293, LTV at 0.0116) are *less* influential than the dynamic, path-dependent, and macro-linked variables — which is precisely why a model that can exploit those richer features, nonlinearly, pulls so far ahead of a plain logistic regression.

A different transition, a different king

Importance is transition-specific. For `current` $\rightarrow$ `30 dpd`, the paper reports that the *number of times the borrower was 30 days late in the past year* dominates all other variables — recent behaviour, not credit score, predicts the near-future slip into delinquency. This is the empirical backing for engineering the rolling delinquency counts, and a reminder that the single shared network has to serve very different importance rankings across the seven destinations at once.

6.5 The standardisation discipline

Continuous inputs (rates, balances, ages, LTV, DTI, FICO) live on wildly different scales, so we standardise them (subtract mean, divide by standard deviation). Heavily skewed quantities like balance are log-transformed first. For a feature column x_j with training-set mean μ_j and standard deviation σ_j ,

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}, \quad \text{or for skewed columns} \quad \tilde{x}_j = \frac{\log(1 + x_j) - \mu_j^{\log}}{\sigma_j^{\log}}. \quad (2)$$

Why standardise at all, and why the log first

Gradient descent struggles when one input ranges over $[300, 850]$ (FICO) and another over $[0, 1]$ (a flag): the loss surface becomes a stretched ravine that the optimiser bounces across slowly. Rescaling every feature to mean 0, variance 1 makes the surface rounder and training faster and more stable. The log on balances pulls in a long right tail (a few million-dollar loans among many small ones) so the post-transform values are roughly symmetric rather than dominated by a handful of giants.

Compute the scaler on the training slice only

It is tempting to standardise the whole dataset once. That leaks: the mean and standard deviation would be computed using future and test data the model is not supposed to have seen. The rule is to fit the scaler on the *training slice only*, for each backtest window, and store it. Baking standardised values into the data lake would be wrong for every rolling window and would quietly contaminate every result.

7 The neural network, built from logistic regression up

This is the modelling core. We build it in five moves, each a small step from the last, so that by the end the deep network feels inevitable rather than magical. Throughout, x denotes the vector of explanatory variables for one loan-month (the engineered features of Section 6), and $\mathcal{U} = \{\text{current}, 30\text{dpd}, \dots, \text{prepaid}\}$ is the set of $K = 7$ states.

7.1 Move 0: what the model must output

For a loan currently in some state, in month $t - 1$, with features $x = X_{t-1}^n$, we want the probability of being in each state u next month:

$$\mathbb{P}[U_t^n = u \mid \mathcal{F}_{t-1}] = h_\theta(u, x), \quad u \in \mathcal{U}, \quad (3)$$

where θ are parameters we will estimate. For this to be a valid probability distribution, the seven numbers $h_\theta(u, x)$ must be non-negative and sum to one. The collection of these conditional

probabilities, for every starting state, *is* the conditional transition matrix — the model’s version of the empirical matrix from Section 4, but now depending on x .

$\mathcal{F}_{t-1}$, the information set

$\mathcal{F}_{t-1}$ is shorthand for “everything known up to month $t - 1$.” Equation (3) says: given all we know today, here is the probability of each state tomorrow. The model is *conditional* on the present, which is what makes it useful for forecasting.

7.2 Move 1: the softmax turns numbers into probabilities

Suppose a model produces seven real-valued “scores” $z = (z_1, \dots, z_K)$, one per state — these can be any real numbers, positive or negative. The **softmax** function squashes them into a probability distribution:

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K. \quad (4)$$

Exponentiating makes every entry positive; dividing by the sum makes them add to one.

Why exponentials?

Softmax rewards bigger scores disproportionately — a score that is 2 larger becomes $e^2 \approx 7.4$ times more probable, not 2 more. This “soft” winner-take-most behaviour is exactly right for classification: the state with the highest score gets most of the probability mass, but the others keep a sliver. It is the smooth, differentiable cousin of just picking the maximum, and differentiability is what lets us train by gradient descent.

7.3 Move 2: logistic regression is softmax of a straight line

The simplest model feeds a *linear* function of the features into the softmax:

$$h_\theta(u, x) = \text{softmax}(Wx + b)_u, \quad W \in \mathbb{R}^{K \times d_x}, \quad b \in \mathbb{R}^K. \quad (5)$$

This is *multinomial logistic regression* — the workhorse of the empirical mortgage literature. Here $\theta = (W, b)$. It is the **zero-hidden-layer** network, and it will be our linear straw man throughout.

Its fatal limitation in one line

The output $h_\theta(u, x)$ can only bend in the directions encoded by the rows of W — it is a fixed linear function of x passed through a fixed squashing. It cannot represent the S-shaped incentive curve, the FICO plateau, or the way two variables *interact*. For prepayment, where those nonlinearities are strongest, it is structurally mis-specified. Everything below is a way to relax exactly this limitation.

7.4 Move 3: from fixed features to *learned* features

A standard fix is to first transform x through some basis functions $\phi(x) = (\phi_1(x), \dots, \phi_{d_\phi}(x))$ — polynomials, splines, step functions — and run logistic regression on those: $h_\theta(u, x) = \text{softmax}(W\phi(x) + b)_u$. The catch: *you have to choose ϕ in advance*, and if your guess is wrong, the model fits poorly. With dozens of mortgage variables interacting in unknown ways, guessing the right basis by hand is hopeless.

The neural-network idea

Instead of fixing the feature functions ϕ ahead of time, *learn them from the data*. Replace ϕ with a *parameterised* transformation ϕ_θ whose parameters are estimated alongside W and b . A neural network is precisely a flexible, trainable ϕ_θ stacked under a final softmax.

7.5 Move 4: stacking layers

A neural network builds ϕ_θ by composing simple operations called *layers*. Each layer takes the previous layer's output, applies (i) a linear map, then (ii) a simple element-wise nonlinearity σ . Writing $h_{\theta,0}(x) = x$ for the inputs, the output of layer ℓ is

$$h_{\theta,\ell}(x) = \sigma\left(W_\ell^\top h_{\theta,\ell-1}(x) + b_\ell\right), \quad \ell = 1, \dots, L-1, \quad (6)$$

with $W_\ell \in \mathbb{R}^{d_{\ell-1} \times d_\ell}$ and $b_\ell \in \mathbb{R}^{d_\ell}$. The final layer applies the softmax instead of σ , producing the seven probabilities:

$$h_\theta(u, x) = \text{softmax}\left(W_L^\top h_{\theta,L-1}(x) + b_L\right)_u. \quad (7)$$

The full parameter vector is $\theta = (W_1, \dots, W_L, b_1, \dots, b_L)$. The layers strictly between input and output are the *hidden* layers; a network with $L-1$ hidden layers and $L=1$ (no hidden layers) collapses back to logistic regression (5). Each hidden layer's output becomes the “learned basis functions” for the next.

Hidden layer, unit, activation

A *hidden layer* is an intermediate stage of computation you never directly observe — its outputs are internal features the network invents. Each coordinate of a layer is a *unit* (or neuron): one affine combination of the previous layer followed by the nonlinearity σ , called the *activation function*. “Depth” is the number of layers; “width” is the number of units per layer.

Zooming in on a single unit makes (6) concrete. Unit k of layer ℓ computes one number from the whole previous layer:

$$(h_{\theta,\ell}(x))_k = \sigma\left(\underbrace{\sum_j (W_\ell)_{jk} (h_{\theta,\ell-1}(x))_j}_{\text{a weighted vote of the previous layer}} + (b_\ell)_k\right). \quad (8)$$

Each weight $(W_\ell)_{jk}$ says how strongly feature j of the previous layer feeds unit k ; the bias shifts the threshold; the activation decides whether the unit “fires.”

7.6 The activation: ReLU

The element-wise nonlinearity we use is the **rectified linear unit**:

$$\sigma(x) = \text{ReLU}(x) = \max(0, x). \quad (9)$$

It passes positive values through unchanged and zeroes out negatives. The paper found ReLU gave both better accuracy and faster convergence than the classic sigmoid $\sigma(x) = 1/(1 + e^{-x})$.

Why a kink is enough

ReLU looks almost laughably simple — a hinge. But stacking many hinged units lets the network assemble arbitrarily intricate piecewise-linear surfaces, the way many short straight segments can trace any curve. The flat-zero region also makes gradients cheap and stable to propagate, which matters when you are taking hundreds of millions of gradient steps. Crucially, *without* a nonlinearity, stacking linear layers would just give one big linear map — no expressive gain at all. The nonlinearity is what makes depth worth having.

7.7 Our concrete architecture

Cross-validation in the paper selected **five hidden layers** of widths

$$200 \rightarrow 140 \rightarrow 140 \rightarrow 140 \rightarrow 140,$$

each with ReLU, feeding a final softmax over the $K = 7$ states. So a single loan-month's feature vector x flows:

$$x \in \mathbb{R}^{d_X} \xrightarrow{W_1, b_1, \text{ReLU}} \mathbb{R}^{200} \xrightarrow{\text{ReLU}} \mathbb{R}^{140} \xrightarrow{\text{ReLU}} \mathbb{R}^{140} \xrightarrow{\text{ReLU}} \mathbb{R}^{140} \xrightarrow{\text{ReLU}} \mathbb{R}^{140} \xrightarrow{\text{softmax}} \Delta^6,$$

where Δ^6 is the set of probability distributions over the seven states.

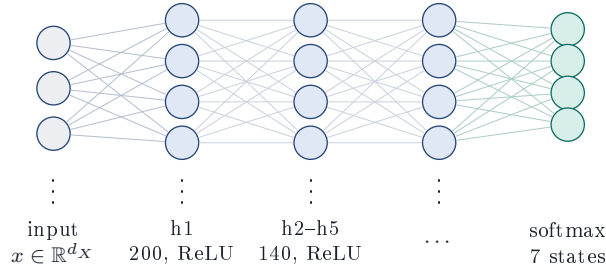


Figure 7: The architecture. The feature vector enters on the left; each hidden layer applies a linear map then a ReLU (equations (6), (9)); the final teal layer applies the softmax (4) to emit a probability over the seven states. Widths are 200, 140, 140, 140, 140; only a few representative units and edges are drawn.

Counting the parameters

A layer mapping $d_{\ell-1}$ inputs to d_ℓ outputs has $d_{\ell-1} d_\ell$ weights plus d_ℓ biases. With an input dimension of, say, $d_X \approx 272$ engineered features (the paper's figure), the count is:

$$\begin{aligned} \text{layer 1: } & 272 \times 200 + 200 = 54,600, \\ \text{layer 2: } & 200 \times 140 + 140 = 28,140, \\ \text{layers 3-5: } & 3 \times (140 \times 140 + 140) = 59,220, \\ \text{output: } & 140 \times 7 + 7 = 987, \\ \text{total: } & \approx 1.4 \times 10^5 \text{ parameters.} \end{aligned}$$

On the order of a hundred thousand free parameters — “tens of thousands,” in the paper's words, and the exact figure scales with how many features the input encoding produces. Fitting that many parameters is only possible because we have on the order of a billion training examples.

7.8 Why depth, and why one network for all transitions

Depth buys expressiveness cheaply

A network with enough units can approximate *any* continuous function on a bounded region (the universal approximation theorem) — including the products and ratios of variables that encode *interactions*, where one variable’s effect depends on another’s value. But there are two ways to add capacity: more units per layer, or more layers. Deep networks (three or more hidden layers) can represent some functions with *exponentially fewer* units than a shallow one, because later layers reuse the features built by earlier layers. The paper’s fitting experiments showed a clear preference for depth, which is itself evidence that mortgage behaviour is genuinely, deeply nonlinear.

The universal approximation theorem, stated

Formally: for any continuous function f on a compact set $\mathcal{K} \subset \mathbb{R}^d$ and any tolerance $\varepsilon > 0$, there is a single-hidden-layer network h_θ (with enough units and a non-polynomial activation) such that $\sup_{x \in \mathcal{K}} |f(x) - h_\theta(x)| < \varepsilon$. So even *one* hidden layer is, in principle, enough to approximate anything — the practical catch is that the required width can be astronomically large, whereas *stacking* layers reaches the same accuracy with far fewer parameters. Depth is an efficiency argument, not an existence one.

One shared network models every transition

We do *not* train a separate model for “current $\rightarrow$ prepaid,” another for “30 dpd $\rightarrow$ current,” and so on. A single network takes the features (including the current state, encoded as input) and outputs the whole next-state distribution at once. Why? Because different transitions lean on the *same* underlying nonlinear features — the forces driving **current** $\rightarrow$ **prepaid** and **30dpd** $\rightarrow$ **prepaid** overlap heavily. Sharing one network means all the data helps estimate those shared features, which is both more parsimonious and statistically stronger than fitting seven separate models.

8 Training: maximum likelihood is cross-entropy

We have a model h_θ that outputs probabilities. How do we choose θ ? By the principle of *maximum likelihood*: pick the parameters that make the states we actually observed as probable as possible.

8.1 The likelihood, and why it collapses to a clean sum

We observe, for N loans over months $t = 0, \dots, T$, the states U_t^n and the features X_t^n . Two standard, mild assumptions make the likelihood tractable:

1. **Exogeneity:** the features (rates, prices, borrower attributes) evolve according to their own laws, not influenced by the parameter θ that governs loan states. So we can treat them as given.
2. **Conditional independence:** given the information at $t - 1$, the next-month states of different loans are independent. (Their *shared* dependence on common covariates is already captured *through* X — that is how the model encodes loan-to-loan correlation.)

Under these, the log-likelihood of all observed states factors into a sum over every loan and every

month:

$$\mathcal{L}_{T,N}(\theta) = \sum_{n=1}^N \sum_{t=1}^T \log h_{\theta}(U_t^n, X_{t-1}^n). \quad (10)$$

The maximum-likelihood estimator is then $\hat{\theta} \in \arg \max_{\theta} \mathcal{L}_{T,N}(\theta)$.

Reading equation (10) aloud

For each loan-month, take the probability the model assigned to the state that *actually happened* next, and take its logarithm. Sum those logs over all loan-months. Good parameters are those that put high probability on the events that really occurred. The double sum is just “every loan, every month” — a billion-ish terms.

8.2 Negative log-likelihood = cross-entropy loss

For optimisation we minimise the *negative average* log-likelihood:

$$\text{loss}(\theta) = -\frac{1}{M} \sum_{(n,t)} \log h_{\theta}(U_t^n, X_{t-1}^n), \quad (11)$$

where M is the number of loan-months. This quantity has another name in machine learning: the **cross-entropy loss**.

They are the same object, seen from two angles

Write the true next state as a one-hot vector y (a 1 in the realised state’s slot, 0 elsewhere) and the model’s prediction as $p = (p_1, \dots, p_K)$. The cross-entropy between them is $-\sum_k y_k \log p_k$. Because y is one-hot, that sum has exactly one surviving term: $-\log(\text{probability the model gave the true class})$. Averaged over all examples, that *is* equation (11). So “maximise likelihood,” “minimise cross-entropy,” and “minimise negative log-likelihood” are three names for one optimisation. This is also the exact number we compare against the empirical-transition-matrix benchmark.

8.3 The gradient that makes it all work

There is a small miracle hiding in the pairing of softmax and cross-entropy. Let $z = W_L^T h_{\theta,L-1} + b_L$ be the final-layer scores (“logits”), $p = \text{softmax}(z)$ the predicted probabilities, and y the one-hot true label. The gradient of the loss with respect to the logits is astonishingly clean:

$$\frac{\partial \text{loss}}{\partial z_k} = p_k - y_k. \quad (12)$$

Where $p - y$ comes from, and why it is beautiful

The loss for one example is $-\sum_k y_k \log p_k$ with $p_k = e^{z_k} / \sum_j e^{z_j}$. Differentiating the log-softmax gives $\partial \log p_i / \partial z_k = \mathbf{1}[i = k] - p_k$. Multiplying by $-y_i$ and summing over i (and using $\sum_i y_i = 1$) collapses to exactly $p_k - y_k$. Read it aloud: the error signal at each output is simply *predicted probability minus actual outcome*. If the model already nailed it ($p = y$), the gradient is zero and nothing changes; if it over-predicted a state, that state’s score is pushed down in proportion to the overshoot. This is why softmax and cross-entropy are not an arbitrary pairing but the natural one: they make the output-layer gradient trivially cheap and perfectly interpretable.

A second reading: cross-entropy is distance from the truth

Minimising cross-entropy is equivalent to minimising the Kullback–Leibler divergence from the model’s predicted distribution to the true conditional distribution of next states. In other words, training does not just “fit labels” — it pushes the whole predicted seven-way distribution as close as possible (in an information-theoretic sense) to reality. That is exactly the object an MBS analyst cares about: not a single guess, but a calibrated distribution over what each loan will do.

8.4 Backpropagation in one equation

Equation (12) gives the gradient at the *output*. To train the earlier layers we propagate it backward. Writing δ_ℓ for the error signal at layer ℓ , the chain rule gives the recursion

$$\delta_{\ell-1} = (W_\ell \delta_\ell) \odot \sigma'(z_{\ell-1}), \quad (13)$$

where $\odot$ is element-wise multiplication and σ' is the activation’s derivative (for ReLU, simply 1 where the unit was active and 0 where it was off). The gradient for each weight matrix is then $\delta_\ell h_{\theta,\ell-1}^\top$.

What backprop is really doing

Read (13) right to left: start with “predicted minus actual” at the output, push it back through the transpose of each layer’s weights (“how much did this unit contribute?”), and gate it by whether each ReLU was on. One forward pass to compute predictions, one backward pass to compute every one of the $\sim 10^5$ gradients — that is the whole engine, repeated for every minibatch. ReLU’s flat derivative is part of why it trains faster than the sigmoid: the gradient passes through active units undiminished instead of being repeatedly squashed.

8.5 Minibatch stochastic gradient descent

We cannot compute the gradient of (11) over a billion examples at once. Instead we use **mini-batch SGD with momentum**: repeatedly draw a small random batch, compute the loss gradient on just that batch (via backpropagation), and step the parameters downhill.

Backpropagation, in one sentence

Backpropagation is the chain rule applied layer by layer from the output back to the input, so that one backward pass yields the gradient of the loss with respect to every one of the $\sim 10^5$ parameters at once.

A momentum update keeps a running average of recent gradients so the path to the minimum is smoother and faster:

$$v \leftarrow \mu v - \eta \nabla_\theta \text{loss}_{\text{batch}}(\theta), \quad \theta \leftarrow \theta + v, \quad (14)$$

with momentum coefficient μ and learning rate η . The paper’s chosen settings are worth recording because we will mirror them:

- **batch size** 4000 loan-months;
- **epoch** ≈ 1.5 million samples, i.e. about 375 gradient steps per epoch;

- **learning-rate schedule** that decays as training proceeds,

$$\eta_t = \frac{\eta_0}{1 + t/800}, \quad \eta_0 = 0.1,$$

with t the epoch number and a half-life of 800 epochs.

Why batches must be drawn at random — and why our shards exist

If a minibatch came from a single quarter or region, its gradient would be *biased* toward that slice, and training could wander or stall. The batches must look like random draws from the whole dataset. That is precisely the job of the shard machinery from Section 5: shuffle shard order, load one RAM-sized shard, shuffle within it, yield batches. The paper solved the same problem with a hash table scattering loans into random folders — our loan-keyed hash is the same idea in modern dress.

8.6 Keeping a flexible model honest: three guards against overfitting

Deep networks are low-bias, high-variance — they can memorise noise. Three standard techniques, all of which we adopt, keep them honest:

1. **ℓ_2 regularisation.** Add a penalty to the loss, giving the trained objective

$$J(\theta) = \text{loss}(\theta) + \lambda \|\theta\|_2^2,$$

which discourages large weights and thus overly wiggly fits. The strength λ is itself cross-validated.

2. **Dropout.** During training, multiply each hidden unit by an independent Bernoulli($1-p$) mask, randomly switching off a fraction p on each step. This stops units from co-adapting into brittle, fictitious partnerships, and is equivalent to training a vast ensemble of thinner networks that share weights; at test time the weights are scaled so the expected activation matches.
3. **Ensembling.** Train eight independently initialised networks on bootstrapped samples and *average their probability outputs*, $\bar{h}(u, x) = \frac{1}{8} \sum_{m=1}^8 h_{\theta(m)}(u, x)$. Each lands in a different local optimum; averaging cancels their idiosyncratic errors and lowers variance. The paper found eight networks captures most of the benefit; beyond that the gains are marginal and the cost is linear.

8.7 The strictly temporal split (and our open cutoff question)

Train on the past, test on the future

The train/validation/test split is *temporal*, never random in time. The paper trains on everything before May 2012, tunes hyperparameters on May–October 2012, and tests on November 2012 through May 2014. Continuous variables are standardised using *training-set* statistics only. This is the only honest way to estimate how the model will perform on months it has never seen.

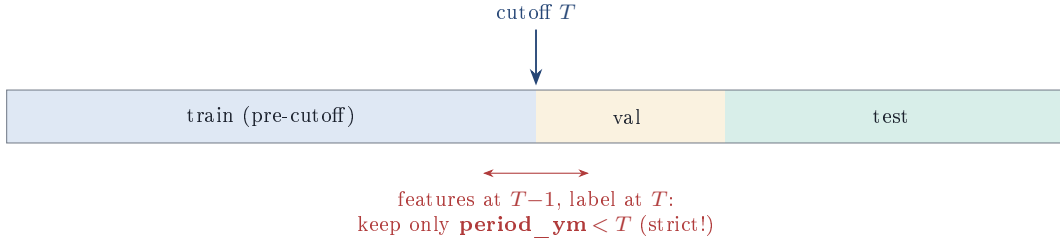


Figure 8: The strictly temporal split. Time runs left to right; the model only ever trains on months before the cutoff and is tested on later months. Because each label looks one month ahead, an example is admissible only when `period_ym < T` — the strict inequality that prevents the post-cutoff transition from leaking into training.

Our cutoff is a genuine design decision — and a question for the advisor

Our Fannie Mae data runs roughly 2000–2025, which means our test window can straddle the COVID shock of 2020–2021, when prepayment and forbearance behaviour was historically abnormal. A reasonable first choice is to train on pre-2022 data, but the COVID distortion makes the cutoff a real judgement call rather than a mechanical one: train through the shock and the model may overfit a once-in-a-generation regime; stop before it and the model never sees a stress episode. The natural answer is a *rolling backtest* — repeatedly retrain up to year Y , test on $Y+1$, absorb $Y+1$, advance — using the leakage-safe `period_ym < cutoff` mask. This is exactly the kind of point to settle with Justin rather than to decide silently.

9 Loan-level results: does depth actually help?

Before the pool, we check the model one loan-month at a time. Two diagnostics matter: overall fit (cross-entropy loss) and discrimination on specific transitions (AUC).

9.1 Goodness of fit: the cross-entropy ladder

The paper reports the out-of-sample loss (11) as the network deepens. Lower is better; this is the same loss we benchmark against the empirical transition matrix.

Table 3: Out-of-sample loss (negative average log-likelihood) by network depth, from the original paper. The ensemble is eight five-layer networks.

Model	In-sample	Out-of-sample (no dropout)	Out-of-sample (with dropout)
0 hidden layers (logistic)	0.1840	0.1805	0.1805
1 hidden layer	0.1680	0.1700	0.1685
3 hidden layers	0.1644	0.1679	0.1671
5 hidden layers	0.1639	0.1684	0.1670
7 hidden layers	0.1638	0.1688	0.1673
Ensemble (8×5-layer)	0.1640	0.1659	0.1654

How to read this table

Two lessons. First, depth helps but with diminishing returns: the big jump is from the linear model (0.1805) to any network at all, and the out-of-sample loss is lowest around *five* layers with dropout — deeper is not automatically better, because capacity also invites overfitting (note the characteristic U-shape). Second, the ensemble beats every single network: averaging eight five-layer nets gives the best out-of-sample loss of all. That is why the project’s target architecture is “five layers, ensembled by eight.”

9.2 Discrimination by transition: AUC

Cross-entropy is one global number; AUC zooms in on individual transitions.

ROC and AUC

Treat a single transition (say **current** $\rightarrow$ **prepaid**) as a yes/no classification: will this loan prepay next month or not? As you sweep the decision threshold, the model trades off true positives against false positives — that trade-off curve is the ROC. The *area under* it (AUC) is a single score in $[0.5, 1]$: 0.5 is a coin flip, 1 is perfect. Equivalently, AUC is the probability the model scores a true prepayer above a true non-prepayer.

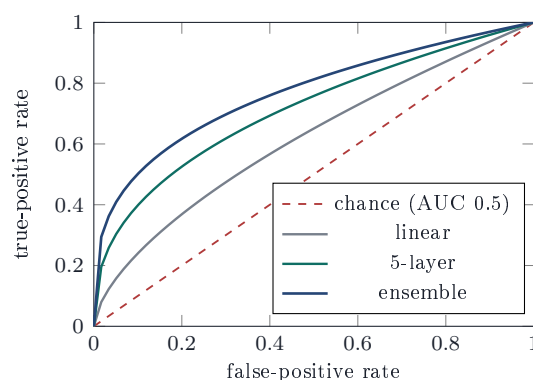


Figure 9: Illustrative ROC curves for **current** $\rightarrow$ **prepaid** (schematic, after the paper’s Figure 17). A curve that bows further toward the top-left corner has higher AUC. The ensemble dominates the single network, which dominates the linear model, which sits only a little above the chance diagonal.

Table 4: Out-of-sample AUC for transitions to prepaid, by depth (original paper). “dd” = days delinquent, “F” = foreclosure, “P” = paid off/prepaid.

Model	C $\rightarrow$ P	30dd $\rightarrow$ P	60dd $\rightarrow$ P	90+dd $\rightarrow$ P	F $\rightarrow$ P
0 hidden layers	0.65	0.77	0.68	0.59	0.57
1 hidden layer	0.72	0.79	0.71	0.76	0.68
3 hidden layers	0.74	0.81	0.73	0.79	0.72
5 hidden layers	0.74	0.81	0.73	0.79	0.73
Ensemble	0.76	0.83	0.74	0.79	0.74

Where the nonlinearity hides

Look at 90+dpd → prepaid: the linear model scores a near-useless 0.59, while a single hidden layer jumps to 0.76 — a 30% improvement. The deeper into delinquency, the more nonlinear the prepayment behaviour, and the more the network adds over logistic regression. This is the loan-level fingerprint of the same nonlinearity that will make the pool-level results so different between the two models.

9.3 What the paper actually found (the headline results)

Beyond the loss and AUC tables, three substantive findings emerge from the paper's analysis, and they are the ones our replication is really testing.

Three facts the paper established

1. **Balance and dynamic variables dominate prepayment.** Loan-balance variables outrank the classic static characteristics; the refinancing incentive and house-price changes matter more than FICO or LTV (the importance table in Section 6).
2. **Borrower behaviour is highly path-dependent.** For delinquency transitions, the recent count of missed payments dominates — standard loan and borrower characteristics are *less* predictive of delinquency than the prior literature assumed.
3. **Unemployment's role is real.** The paper firmly establishes the economic significance of state unemployment for transitions, a variable earlier (smaller-sample) studies had dismissed — evidence that common macro factors drive correlated, pool-level risk. These are exactly the claims a flexible model can express and a linear one cannot, and they set up the pool-level payoff in Section 10.

Where our own numbers will go

The tables in this section report the *paper's* published results — the benchmark our replication is measured against, not our own fitted numbers (those are still being computed on the Fannie Mae panel). When the rolling backtest completes, our loss-by-depth, AUC-by-origin, and reliability results drop straight into these same exhibits as additional columns, and the gradient-boosted-tree challenger of Section 11 joins them as one more. The structure is built to receive the numbers; only the cells are pending.

9.4 What we realistically expect to replicate

Honest expectations

Our absolute loss numbers will not match the paper's to the third decimal, and they should not: different dataset (Fannie Mae, not CoreLogic), different and initially smaller feature set (macro covariates are out of scope for the first pass), a different and more recent time span. What we expect to reproduce is the *shape* of the findings: networks beat logistic regression and the empirical-matrix benchmark; five layers ensembled is near the sweet spot; and the gains concentrate in prepayment and in transitions out of severe delinquency. The qualitative story is the replication; the exact digits are not.

10 Pool-level prediction: the result that matters most

This is the section to read twice. Everything so far has been at the level of a single loan-month. Now we aggregate to a *pool* — a thousand loans bundled together — which is the object that mortgage-backed securities are written on. The punchline of the whole dissertation lives here: *the nonlinearities that help a little per loan compound into a large, decisive advantage at the pool level.*

10.1 Why pools, and why investors care

MBS, and pool-level risk

A mortgage-backed security (MBS) bundles many mortgages and sells investors the right to the combined cash flows. The investor does not care about any one borrower; they care about how many loans in the pool *prepay* (returning principal early, to be reinvested at lower rates) or *default*. Pool-level prepayment and delinquency rates are exactly what drive MBS value. So a model that forecasts the *pool* accurately is directly a model that prices MBS accurately.

10.2 The crux: a pool prediction is a sum, not a new model

There is no pool-level model

The expected behaviour of a pool is just the sum of the expected behaviours of its loans. For a pool of N loans and a 12-month horizon,

$$\underbrace{\text{predicted \# prepayments}}_{\text{pool-level}} = \sum_{i=1}^N \mathbb{P}(\text{loan } i \text{ prepays within 12 months}), \quad (15)$$

where each loan's probability comes from the *same* loan-level transition network, rolled forward 12 months. We never fit a separate pool model. Pool predictions are a pure cross-sectional sum of loan-level outputs.

Why this is the whole game

Because the pool prediction is built by addition, two things follow immediately. Accuracy is inherited: a better loan-level model gives a better pool-level model for free. And *bias* is inherited too: if the loan-level model systematically over-predicts prepayment, those errors do not cancel across loans — they pile up in the sum. That second fact is precisely why the linear model fails at the pool level, as we will see.

10.3 The roll-forward, explained slowly

Equation (15) needs “the probability loan i prepays within 12 months.” Our network only directly predicts *one month ahead*. So we must roll it forward. This is the step worth pinning down precisely.

Step 1 — one month is a matrix. For loan i in month m , the network turns its features into a full next-state distribution for *every* possible current state. Collect these into a 7×7 conditional transition matrix $P_m^{(i)}$, whose (u, v) entry is

$$(P_m^{(i)})_{u,v} = h_{\theta}(v \mid u, x_m^{(i)}) = \mathbb{P}(\text{state } v \text{ next month} \mid \text{state } u \text{ now}).$$

Each row is a probability distribution (it sums to 1). **prepaid** and **REO** are *absorbing*: their rows put all mass on themselves.

Step 2 — represent “where the loan is” as a vector. Let $\pi_0^{(i)}$ be a length-7 one-hot row vector with a 1 in the loan’s current state. Multiplying a state distribution by the transition matrix advances it one month:

$$\pi_1^{(i)} = \pi_0^{(i)} P_0^{(i)}.$$

Step 3 — chain twelve of them. Twelve months ahead is twelve matrix multiplications:

$$\pi_{12}^{(i)} = \pi_0^{(i)} P_0^{(i)} P_1^{(i)} \cdots P_{11}^{(i)}. \quad (16)$$

The probability loan i has prepaid by month 12 is simply the **prepaid** entry of $\pi_{12}^{(i)}$ — and because **prepaid** is absorbing, that entry is exactly the cumulative chance of *ever* prepaying within the year.

Answering the roll-forward question directly

You do **not** ask the network for a 12-month probability in one shot. You ask it for *this* month, get a 7×7 matrix, step the state vector forward, and repeat twelve times. Twelve one-month predictions chained together *are* the twelve-month prediction. Equation (16) is the entire roll-forward.

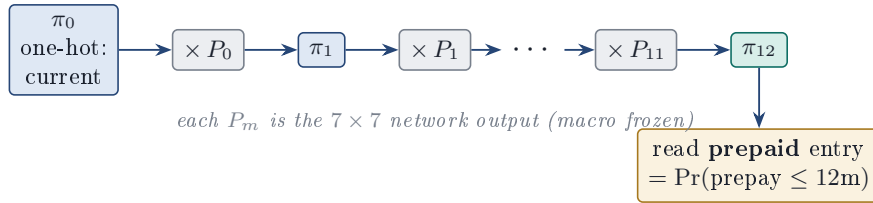


Figure 10: The roll-forward, the step that the loan-level network does not do directly. Start from a one-hot state vector π_0 , multiply by twelve successive one-month transition matrices $P_0, \dots, P_{11}$ (each produced by the network from the loan’s features that month, with macro variables frozen), and read off the **prepaid** coordinate of π_{12} . Because **prepaid** is absorbing, that coordinate is the cumulative probability of having prepaid within the year.

This chaining is an instance of the *Chapman–Kolmogorov* identity for Markov chains: the t -step transition matrix is the product of the one-step matrices,

$$P^{(0 \rightarrow t)} = P_0 P_1 \cdots P_{t-1}, \quad (17)$$

and the loan’s t -month state distribution is $\pi_t = \pi_0 P^{(0 \rightarrow t)}$.

Why multiplying matrices advances time

Entry (u, v) of the product $P_0 P_1$ is $\sum_w (P_0)_{u,w} (P_1)_{w,v}$ — the probability of going $u \rightarrow w$ in month 0 *and then* $w \rightarrow v$ in month 1, summed over every possible intermediate state w . That sum-over-paths is exactly the law of total probability across the unobserved middle state, and iterating it twelve times sums over all twelve-month trajectories at once. So a single matrix product silently accounts for every way a loan could wander through delinquency and back before (or instead of) prepaying.

What changes between the twelve months? Two options:

1. **Frozen covariates (the cheap, accurate-for-12-months choice).** Freeze the macro variables (rates, prices, unemployment) at their month-0 values. Only the loan's own deterministic features evolve — its scheduled balance, its age, its remaining term — and its state, which is what the π vector tracks. Then the matrices in (16) are deterministic and you just multiply them. This is how the paper produces the point predictions in the scatter plots.
2. **Simulated covariates (for the full distribution).** Treat the macro variables as random, simulate many future paths, recompute the matrices along each path, and average. This is more expensive but yields a whole *distribution* of pool outcomes, not just a mean (next subsection).

Why freezing is legitimate over a year

Over twelve months, a borrower's state can swing a lot, but the national mortgage rate and local prices move comparatively slowly. Freezing them introduces only a small error at the one-year horizon while removing the need to model their joint dynamics — a very favourable trade for the point estimate. For the *distribution*, though, the spread is mostly driven by where rates go, so there we must let the rate move.

10.4 Building the pools

To stress-test the model across risk levels, the paper builds pools that deliberately differ in quality. Take 2 million test loans, **sort** them by a chosen characteristic, and chop the sorted list into consecutive blocks of 1,000 — giving 2,000 pools that range smoothly from low to high risk. Four sorting keys are used:

FICO score • original interest rate • LTV ratio • predicted probability of staying current.

(Our project mirrors the first three.) For each pool, the predicted prepayment count is the sum (15) of its 1,000 loans’ rolled-forward probabilities, and the observed count is what actually happened in the test data over the next 12 months.

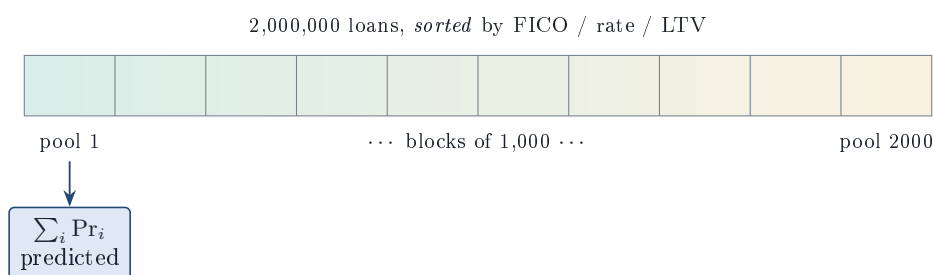


Figure 11: *Building pools of graded risk. Sort all loans by a characteristic, then chop the sorted list into consecutive blocks of 1,000. Pools then range smoothly from low to high risk, and each pool’s prediction is just the sum of its loans’ rolled-forward probabilities.*

10.5 The headline picture: observed vs. predicted

Plot each pool as a point: observed prepayments on the x -axis, predicted on the y -axis. Perfect prediction lies on the 45-degree line $y = x$.

What the scatter shows — and it is striking

The five-layer network’s points hug the $y = x$ line across the whole range of pool risk. The linear zero-layer model’s points sit *systematically above* the line — it persistently over-predicts prepayment, and the bias does not wash out. The difference is not subtle, and it is the visual heart of the dissertation.

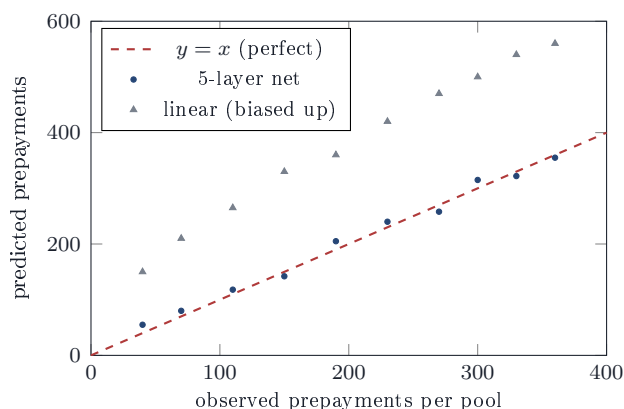


Figure 12: Out-of-sample pool predictions (schematic, after the paper’s Figures 19–22). Each point is one pool: observed prepayments on the x -axis, model-predicted on the y -axis. The network’s points track the perfect-prediction line; the linear model’s points lie consistently above it — a bias that, crucially, does not cancel out across pools.

Table 5: Out-of-sample pool-level distribution, averaged over 50 test portfolios of 10,000 loans each, one-year horizon (original paper, Table 22). “Standardised gap” is the prediction error in units of the forecast’s own standard deviation.

	0-layer (linear)	5-layer network
Avg. actual prepayments	1723.8	1723.8
Avg. predicted prepayments	2853.8	1456.9
Avg. absolute gap	1186.0	278.5
Avg. standardised gap	2.4	1.9

Reading Table 22 — the single most quotable result

The truth is about **1724** prepayments per pool. The linear model predicts **2854** — a 1186-prepayment error, two-thirds too high. The five-layer network predicts **1457** — an error of 279, more than *four times* smaller. The network is not perfect (it slightly under-predicts here), but it is dramatically closer, and its forecast distribution is tighter too. For an MBS investor pricing cash flows, that is the difference between a usable model and a misleading one.

10.6 The full distribution, via Monte Carlo on rates

For a richer view, the paper simulates the *distribution* of a pool’s prepayments rather than a single number. The recipe:

1. Fit a small time-series model to the national mortgage rate — an AR(4) on the Freddie Mac series (the same family as our PMMS feature).

2. Simulate many future rate paths from that AR(4); freeze the other macro variables.
3. Along each path, recompute every loan's roll-forward (16) and sum to a pool prepayment count.
4. The spread of those counts across paths is the forecast *distribution*.

AR(4)

An autoregressive model of order 4: next period's rate is a linear function of the previous four periods' rates plus noise. It is a compact way to generate plausible random future interest-rate paths so that correlated prepayment — everyone refinancing at once when rates drop — is captured. The paper's fitted coefficients (intercept and four lags) are $[0.6687, 1.3514, -0.5131, 0.2410, -0.0838]$, with the lag length chosen from the partial-autocorrelation plot — a deliberately simple model whose only job is to generate realistic rate scenarios.

The tell-tale sign of a good model

For the five-layer network, the *actually observed* prepayment count lands near the *centre* of the simulated distribution — the model's uncertainty is well-calibrated. For the linear model, the observed count falls out in the *tail* — reality is an event the linear model thought was unlikely. A model that is routinely surprised by reality is the wrong model to price a security with.

When the macro covariates are *frozen*, the loans become conditionally independent and the pool prepayment count $S = \sum_{i=1}^N B_i$ is a sum of independent Bernoulli indicators B_i (loan i prepays or not), with $\Pr(B_i = 1) = p_i$. Its mean and variance are then exactly

$$\mathbb{E}[S] = \sum_{i=1}^N p_i, \quad \text{Var}(S) = \sum_{i=1}^N p_i(1 - p_i), \quad (18)$$

and for a pool of hundreds or thousands of loans the central limit theorem makes S approximately Normal (equivalently, a Poisson approximation applies when the p_i are small).

Why a few hundred loans already give a tight forecast

The mean prediction (18) is the same sum (15) we have used throughout. The new content is the *variance*: it grows like N while the mean also grows like N , so the relative spread $\sqrt{\text{Var}(S)}/\mathbb{E}[S]$ shrinks like $1/\sqrt{N}$. That is why the paper can say closed-form Normal/Poisson approximations are accurate “even for pools with only a few hundred loans”: pooling averages away idiosyncratic loan-level uncertainty. What it does *not* average away is the shared macro risk — which is precisely why the full distribution (previous subsection) must simulate the rate path rather than freeze it.

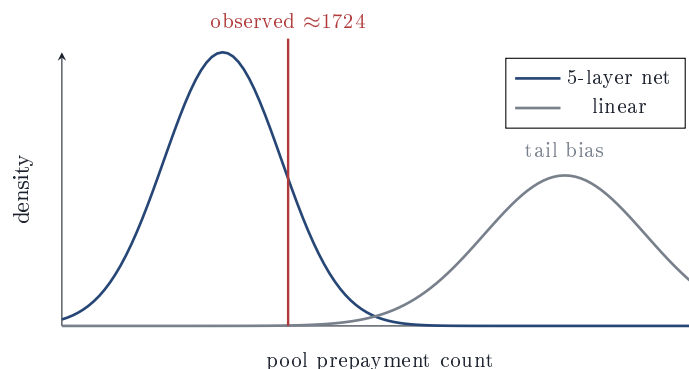


Figure 13: Forecast distributions of pool prepayments (schematic, after the paper’s Figures 23–24 and Table 22). The network’s distribution is narrow and brackets the observed outcome; the linear model’s distribution is shifted far to the right, leaving the truth in its left tail. Numbers echo Table 22: linear mean ≈ 2854 , network mean ≈ 1457 , observed ≈ 1724 .

10.7 Why the linear model is biased and the network is not

The mechanism, spelled out

The pool prediction (15) sums one probability per loan. If each loan’s probability were *unbiased*, the per-loan errors would be roughly independent and would average out — the law of large numbers would make the pool sum very accurate. But the linear model is *mis-specified* for prepayment: it cannot represent the S-shaped incentive curve or the FICO plateau, so it makes errors with a *consistent sign*. Worse, every loan in the pool shares the same macro covariates (one national rate, one local price path), so when the linear model mis-reads that shared environment, it mis-reads it *the same way for every loan at once*. The errors are correlated, they all point the same direction, and the sum amplifies them instead of cancelling them. The network, by fitting the true nonlinear shape, keeps each loan’s probability close to unbiased — so the sum stays accurate. This is the deep reason a small per-loan edge becomes a large per-pool edge.

The dissertation’s thesis, restated at the pool level

Capturing the nonlinearities in borrower behaviour does not just sharpen individual-loan forecasts; it removes a *systematic, correlated bias* from pool-level forecasts. Because MBS are priced off pool-level cash flows, that bias removal translates directly into more accurate MBS valuation — which is the practical pay-off the whole exercise is aiming at.

11 A non-neural challenger: gradient-boosted trees

Everything so far has pitted one flexible model (the neural network) against one rigid one (logistic regression). The network won. But that raises a sharper question the dissertation can answer, and which the original paper never asked: *when the flexible model wins, is it winning because it is flexible, or because it is **smooth**?* To separate those two explanations we add a second flexible learner with the *opposite* inductive bias — gradient-boosted trees — and race it against the net. This section explains what that learner is, why we chose it over the alternatives we considered, where it slots into the machine we have already built, and what each possible outcome of the race would teach us.

11.1 The question GBT is built to answer

Flexibility vs. smoothness

The neural network is *flexible* (it can fit complicated surfaces) *and smooth* (it is biased toward gently-curving functions). Logistic regression is neither. So “the net beats logit” confounds the two virtues. A gradient-boosted tree ensemble is just as flexible as the net but is *not* smooth — it builds its predictions out of sharp, axis-aligned steps. Racing net against GBT therefore holds flexibility roughly fixed and varies smoothness, isolating *which kind of flexibility* the mortgage-transition surface actually rewards.

11.2 What a decision tree is

The atom of the challenger is the **decision tree**. A tree asks a sequence of yes/no questions about the features — “is FICO below 660?”, then “is the rate incentive above 1%?” — and each path down the tree ends in a *leaf* that emits a constant prediction.

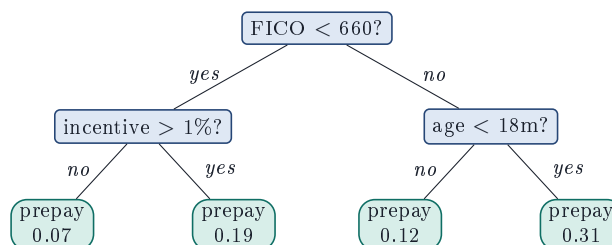


Figure 14: A single (tiny) decision tree. Internal nodes split on one feature at a time; leaves give a constant predicted probability. A path that splits on FICO and then on incentive has automatically captured an interaction between them — the effect of incentive differs depending on the FICO branch.

Mathematically a tree partitions the feature space into rectangular regions $R_1, \dots, R_J$ and predicts a constant c_j in each:

$$f(x) = \sum_{j=1}^J c_j \mathbf{1}\{x \in R_j\}. \quad (19)$$

What trees are naturally good and bad at

Three things come for free: trees are *scale-invariant* (only the order of a feature’s values matters, so no standardisation is needed — contrast equation (2)); they handle missing values and categoricals like origin **state** natively; and they capture interactions automatically by stacking splits. The cost is the shape of (19): the prediction is *piecewise constant*, a landscape of flat plateaus with vertical cliffs between them. A single tree is also high-variance — redraw the data and the splits jump around. Boosting fixes the variance; the piecewise-constant shape is intrinsic, and it is the whole point of the comparison.

11.3 From one tree to a boosted ensemble

A single shallow tree is a weak model. **Gradient boosting** turns many weak trees into a strong one by building them *sequentially*, each new tree correcting the errors the running ensemble has made so far. The model is an additive sum,

$$F_M(x) = \sum_{m=1}^M \nu h_m(x), \quad F_m = F_{m-1} + \nu h_m, \quad (20)$$

where each h_m is a tree and ν is a small *learning rate* that shrinks each tree’s contribution so the ensemble improves in cautious steps.

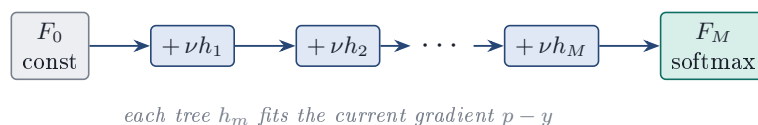


Figure 15: Gradient boosting. Start from a constant, then add one shrunken tree at a time, each fitted to the direction that most reduces the loss. The final ensemble passes through a softmax so its seven outputs are a valid probability distribution — exactly what the roll-forward and cashflow engines require.

The word “gradient” is literal: each tree h_m is fitted to the *negative gradient* of the loss with respect to the current predictions — the direction in which nudging the predictions would most reduce the loss. And here a familiar face returns.

The same $p - y$ gradient as the net

With the multiclass softmax cross-entropy loss — the *same* loss the network minimises — the gradient with respect to each class’s score is once again

$$g_k = p_k - y_k, \quad \text{with curvature (Hessian)} \quad H_{kk} = p_k(1 - p_k),$$

exactly equation (12) from the network’s output layer. So each boosting round fits seven trees (one per class) to the residual “predicted minus actual,” and uses the curvature to set each leaf’s value by a Newton step, $c_j \approx -\sum g / (\sum H + \lambda)$. The profound point: **the net and GBT minimise the identical objective with the identical gradient signal**. They differ in one thing only — the *class of functions* each searches to represent that signal: smooth compositions of ReLU layers for the net, sums of axis-aligned step functions for GBT.

LightGBM, and why this particular implementation

LightGBM is a fast, memory-light gradient-boosting library. Two features make it the practical choice here. It *bins* each continuous feature into a histogram (e.g. 255 bins), so a row stores one byte per feature and the whole thinned training set — tens of millions of rows — fits in a couple of gigabytes of RAM. And it grows trees *leaf-wise* (splitting wherever the loss drops most) rather than level-by-level, reaching lower loss in fewer splits. For a 7-class problem it builds seven trees per round, the source of a roughly 7× wall-clock factor.

Is it actually cheap? The numbers say yes

The “almost free” claim is concrete. LightGBM’s binned dataset stores one byte per feature-value, so our thinned $\sim 74 \text{ M} \times 30$ training slice occupies roughly **2.2 GB** of RAM, versus $\sim 17.8 \text{ GB}$ for a raw float64 copy — it fits on any laptop. On its published benchmark LightGBM trains a $10.5 \text{ M} \times 28$ dataset in about **130 seconds** on 16 cores; scaling for our row count and the 7× multiclass multiplier puts a full window at an estimated ~ 1.5 – 2.5 hours (an extrapolation, flagged as such), so all eleven backtest windows are a single overnight CPU job. And the GPU barely helps: at ~ 30 features LightGBM’s leaf-wise growth is sequential and measured GPU speedups are around 1× (often slower), so the whole leg runs for \$0 on the Apple-Silicon Mac, or a few dollars on a cheap CPU pod — leaving the scarce GPU credit for net re-runs. Compared with training and ensembling

the neural network, the challenger is a rounding error on the compute budget.

11.4 Two flexibilities, opposite biases

We can now see the crux of the whole comparison in a single picture. Take the smooth prepayment S-curve from Section 6. The net, biased toward smoothness, traces it with a gentle curve. GBT, built from steps, can only *approximate* it with a staircase — and to make the staircase fine it needs many splits, paying in variance and, crucially, in the accuracy of the probability *levels* the pool valuation consumes.

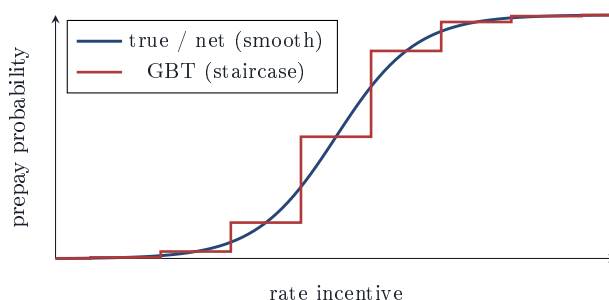


Figure 16: *The inductive-bias contrast in one dimension. A smooth target (the prepayment incentive curve) is traced gracefully by a smooth learner and only stepped through by a tree ensemble. Where the economically important structure is smooth, the staircase is at a disadvantage; where it is genuinely jumpy (policy thresholds, vintage cliffs), the staircase can win. Which effect dominates on this panel is an empirical question.*

The same contrast in two feature dimensions: a tree carves the space into axis-aligned boxes, while the net draws a smooth, possibly diagonal, boundary.

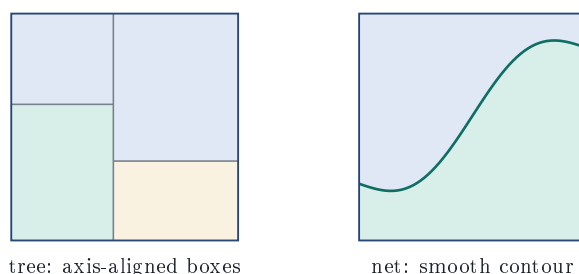


Figure 17: *Decision boundaries over two features (say FICO and incentive). The tree can only cut parallel to the axes, tiling the plane with rectangles; the network can bend a smooth boundary in any direction. Both are flexible; their flexibility has a different grain.*

Same capacity, opposite bias — a controlled experiment

Because the net and GBT have comparable capacity but opposite inductive bias, a net-vs-GBT gap is attributable to the *kind* of flexibility, not the *amount*. That is what upgrades the dissertation’s claim from the generic “flexible beats linear” to the specific and more interesting “the prime-conforming transition surface rewards *smooth* flexibility.”

11.5 Why GBT, and not the other candidates

GBT was not the only non-neural learner we could have added. We considered several and deliberately dispatched them; recording that reasoning is part of the contribution, because the

choice of a single, well-justified challenger is what keeps the comparison clean.

Table 6: *The benchmark brainstorm: candidates considered and the verdict on each.*

Candidate	What it is	Verdict
Random forest	many de-correlated trees, averaged	Dominated by boosting in every relevant credit-ML comparison; at most a weaker tree baseline, not worth a full leg.
GAM / EBM / penalised splines	additive smooth shapes (+ pairwise terms)	Interpretation tools, largely redundant with the augmented-logit rung; the EBM is itself a boosted tree–GAM hybrid. Dispatched.
Kernel SVM / Gaussian process	similarity-based smooth learners	Do not scale to tens of millions of rows. Dispatched.
Elastic-net	ℓ_1/ℓ_2 -shrunk linear model	Cheap, slots into the nested ladder; a legitimate <i>optional</i> extra rung, but light — not a heavy backtest member.
multinomial logit		
Frontier tabular DL (FT-Transformer, TabNet)	attention-based deep tabular nets	Another neural net — we want the field-standard <i>non-neural</i> baseline, not a second net.
GBT (LightGBM)	boosted trees, multiclass softmax	Chosen.

Why GBT is the right single addition

GBT is the unique candidate that is *all four* of: (a) the field-standard strong *non-neural* baseline on tabular data (the *Elements of Statistical Learning* lineage, and the default-to-beat in the modern tabular-ML literature — Shwartz-Ziv & Armon, Grinsztajn et al., Borisov et al., McElfresh et al.); (b) a match for the net on flexibility, so the race isolates inductive bias rather than capacity; (c) a producer of probability *levels* that can survive the pool roll-forward (after calibration, §11); and (d) almost free to run. No other candidate clears all four bars.

11.6 The gap in the literature it fills

A genuine, defensible gap

The paper we replicate (Sirignano et al.) benchmarks its deep network only against linear/logistic specifications — it never reports a tree ensemble. The closest precedent that *does* race nets against gradient boosting on consumer credit (Albanesi & Vamossy, who find a net+GBT *hybrid* best) is binary default on consumer-credit data, with no mortgage-pool valuation. Tree ensembles are the standard ML comparator on exactly this kind of GSE mortgage data (Fuster et al.; Gu, Kelly & Xiu in asset pricing), yet a head-to-head on the full seven-state transition structure *with* a pool-level cashflow evaluation has not been published. That is the gap the extension claims.

11.7 Where it plugs in: reuse, not rebuild

The challenger is cheap to add precisely because it inherits the entire machine. It reads the *same* exported Parquet the net reads, through the same loader, the same window masks and frozen test slices, the same evaluator, and the same pool composition and cashflow engines. The only

genuinely new code is the trainer and a one-line prediction adapter, plus a single model-agnostic seam in the pool engine.

The locked decisions that keep the comparison controlled

Five choices make the race a clean experiment rather than a confound:

1. **A single 7-class softmax model with origin state as a categorical feature** — matching the net’s single-model design, not four per-origin trees. (A single tree model *contains* the four-model solution as a special case — split on origin at the root — but is free to pool structure across origins where the data justify it. We initially considered four separate models and rejected them: that would confound the learner family with the problem decomposition.)
2. **Byte-identical features to the net’s input, minus standardisation** — same columns, same missingness indicators; the scaler is dropped only because trees are scale-invariant.
3. **Horvitz–Thompson importance weights on the training rows only** — they enter LightGBM’s per-row gradient and Hessian natively, keeping the thinned-sample loss an unbiased estimate of the full-sample loss, exactly as the net’s weighted cross-entropy does.
4. **CPU, not GPU** — a documented deviation, because at ~ 30 features LightGBM’s leaf-wise growth is sequential and the GPU barely helps (often $\leq 1\times$); the scarce GPU credit is reserved for net re-runs.
5. **Tune once on the 2015 window, freeze, and roll** — one grid selects the config; it is then looped over all eleven backtest windows, with per-window validation used only for early stopping.

11.8 The one extra step the net did not need: calibration

Calibration, and why GBT may need it

A model is *calibrated* if, among the loans it assigns a 5% prepayment probability, about 5% actually prepay — i.e. its probability *levels* are trustworthy, not just its rankings. Boosting tends toward over-confidence, worst on the rare credit-event classes (into 90+ dpd, foreclosure, REO) that drive cashflows. Because the pool valuation consumes levels, we first check reliability diagrams on the raw GBT outputs; if they sit off the diagonal, we fit a small per-window correction on the validation slice only — *temperature scaling* (one parameter, preserves rankings) first, per-class *isotonic* regression as a fallback — before any levels-consuming step. This mirrors the calibration Fuster et al. apply to tree-based mortgage probabilities, and it is the one methodological step the smooth, naturally better-calibrated net did not require.

11.9 What each outcome would mean (pre-registered)

The experiment is designed so that *every* outcome is informative — which is what makes it a clean contribution rather than a fishing expedition.

- **GBT $\approx$ net (a tie):** *flexibility* is what matters and architecture is secondary — reinforcing the headline that three hidden layers suffice in prime conforming credit, where the original paper needed five on subprime-heavy private-label data.
- **net > GBT:** the *smooth* structure of mortgage behaviour — the incentive S-curve, the seasoning hump, convex FICO decay — rewards smooth learners. This is the sharper, more

publishable claim, since trees fit staircases and the economically important dimensions of the target are smooth.

- **GBT > net:** legitimate, but the tabular-ML metafeatures here (huge N , high size-to-feature ratio, severe class imbalance) all independently favour GBT, so a GBT win would not by itself confirm the smoothness story — attributing it specifically would need the smoothing ablation. We would report it as such rather than over-claim.

The diagnostic is the per-origin breakdown

Whichever way the headline lands, the per-origin-state NLL and AUC breakdown is the real instrument: it shows *where* each learner’s flexibility pays off. We read it especially in the sparse deep-delinquency cells — the **90+dpd** → **foreclosure** transition where the network already showed its largest edge over logistic regression (Section 9). If the net’s advantage there is a smoothness advantage, GBT should fail to match it precisely in those cells.

11.10 Its contribution to the dissertation

What the challenger buys

GBT sharpens the dissertation’s central claim from “flexible beats linear” to a statement about *which kind* of flexibility a prime-conforming mortgage transition surface rewards — a question the original paper did not pose and the credit-ML literature has not answered with a pool-level evaluation. It adds the field-standard non-neural baseline at near-zero compute, turns the nonlinearity result into a controlled experiment on inductive bias, and — through the per-origin breakdown — localises exactly where smooth structure matters. The challenger costs little and, whatever the verdict, pays for itself in interpretation.

12 Our approach: what transfers, what changes, what is open

It helps to separate cleanly what we inherit from the paper, what we deliberately change, and what remains genuinely undecided.

12.1 What transfers wholesale

- The **seven-state monthly transition** formulation and the `state/state_next` target.
- The **model family**: softmax output, ReLU hidden layers, five layers wide 200/140/140/140/140, ensembled by eight.
- The **training recipe**: cross-entropy loss, minibatch SGD with momentum, decaying learning rate, ℓ_2 + dropout + ensembling, strictly temporal split.
- The **pool method**: cross-sectional summation of rolled-forward loan-level probabilities, with pools built by sorting on FICO / rate / LTV.
- The **benchmarks**: the empirical transition matrix and the zero-layer logistic regression as the linear straw man.

12.2 What we change

- **Data source.** Fannie Mae SFLP (public) instead of CoreLogic (proprietary, >120M loans). CoreLogic is not replicable with Fannie Mae data, but the methodology is — which is exactly

why this replication is feasible at all.

- **Macro covariates.** The original uses a rich macro panel down to the zip-code level. Our first pass holds these out of scope; the pipeline is structured so a macro join can be bolted on later without reprocessing. Our early feature set is therefore leaner, and we should expect smaller absolute gains until the macro layer is added.
- **Rate and price series.** We construct the incentive feature from Freddie Mac PMMS and current LTV from the FHFA HPI — public stand-ins for the proprietary series the paper used.
- **Time span.** Roughly 2000–2025, which brings the COVID regime inside the sample.
- **An added non-neural baseline.** Where the paper stops at a linear straw man, we add a gradient-boosted-tree challenger of equal flexibility (Section 11) to turn the nonlinearity result into a controlled test of *which kind* of flexibility the surface rewards.

12.3 What is genuinely open (good questions for the advisor)

- **The train/test cutoff** given the COVID distortion — a fixed pre-2022 split versus a rolling backtest, and how to treat the forbearance-driven anomalies of 2020–2021.
- **Path-dependent features.** The paper found rolling delinquency counts to be among the strongest delinquency predictors; engineering them is a high-value addition our current schema does not yet include (Section 6).
- **Absorbing-state convention** for `state_next` on terminal months, and the exact mapping of ambiguous Zero Balance Codes (e.g. repurchases, reperforming sales) into the seven states.

A note on intellectual honesty

A replication is most valuable when it is candid about where it departs from the original and why. Our results will differ from the paper’s in level; the contribution is showing that the *structure* of the finding — nonlinearity matters, especially for prepayment, and especially at the pool level — survives the move to a different, public dataset and a more recent, more turbulent period. If it survives, that is a genuinely useful external validation of the original claim. If parts of it do not, that is an interesting finding in its own right.

13 How it all fits together

Read top to bottom, the dissertation is one clean arc:

1. **The data** is a billion-row loan-month panel; the engineering challenge is to process it without ever loading it whole (Sections 3, 5).
2. **The target** is the next-month state among seven, derived from delinquency status and the zero-balance code, benchmarked against the empirical transition matrix (Section 4).
3. **The features** encode mortgage economics — refinancing incentive, current LTV, burnout — whose relationships with behaviour are sharply nonlinear (Section 6).
4. **The model** is a softmax-headed neural network, built up from logistic regression by learning its own basis functions through five ReLU layers (Section 7).
5. **Training** is maximum likelihood, equivalently cross-entropy minimisation, by minibatch SGD, guarded against overfitting and split strictly in time (Section 8).

6. **Loan-level**, depth beats the linear model and the benchmark, most decisively on prepayment and severe-delinquency transitions (Section 9).
7. **Pool-level**, the loan-level model is summed — with no new model — and rolled forward twelve months; the network tracks reality while the linear model is systematically and correlatedly biased, which is the result that matters for MBS (Section 10).
8. **The challenger**, a gradient-boosted-tree ensemble of equal flexibility but opposite (staircase) inductive bias, is raced against the net to ask not just whether flexibility helps but whether *smooth* flexibility is what the surface rewards (Section 11).

The one-line takeaway

Predict each loan well with a model flexible enough to capture the nonlinearities, then *add up* the loans — and you get pool-level mortgage forecasts good enough to price the securities built on them. Reproducing that chain on public Fannie Mae data, end to end, is the dissertation.